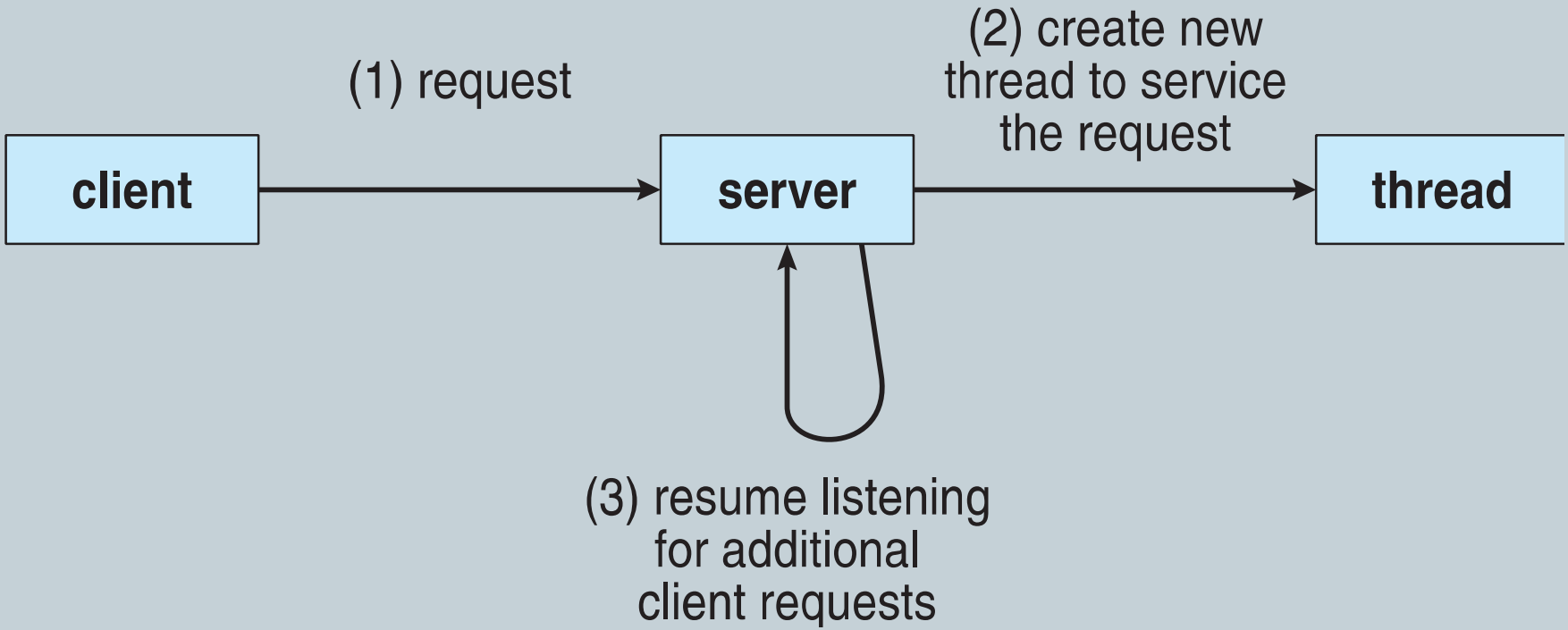
Threads & Concurrency



Multithreaded Server Architecture



Benefits



- **Responsiveness**
- **Resource Sharing**
- **Economy**
- **Scalability**

Multicore Programming



- Challenges are:
  - Dividing activities
  - Balance
  - Data splitting
  - Data dependency
  - Testing and debugging

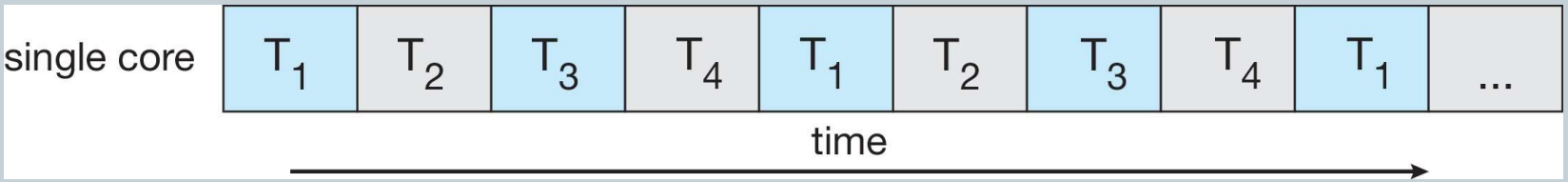
Multicore Programming



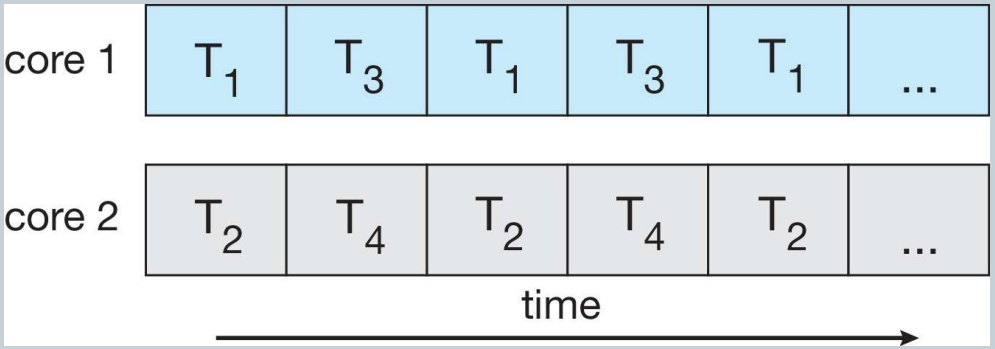
- Types of parallelism
  - Data parallelism
  - Task parallelism

Concurrency vs. Parallelism

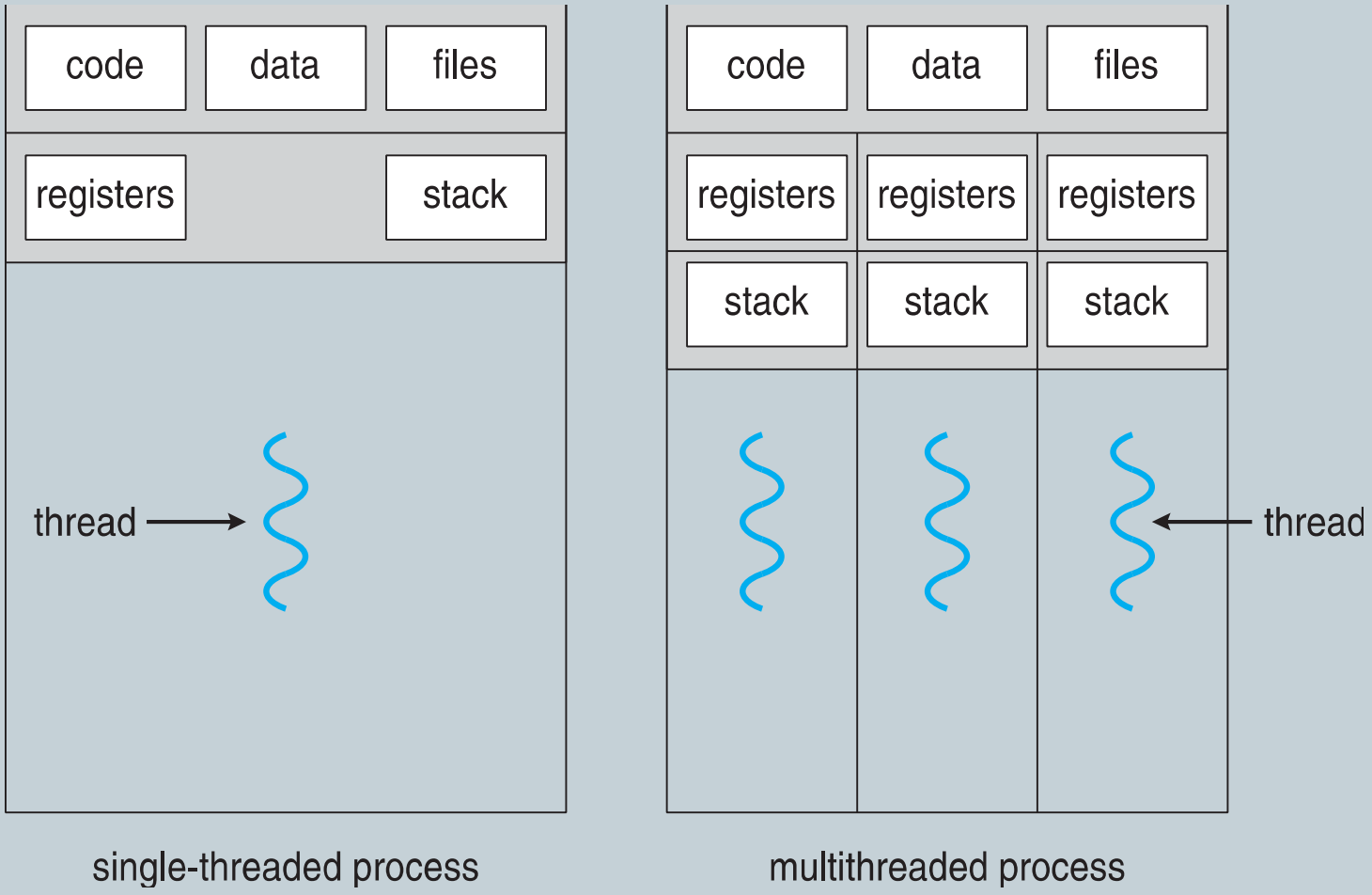
- Concurrent execution on single-core system:



- Parallelism on a multi-core system:

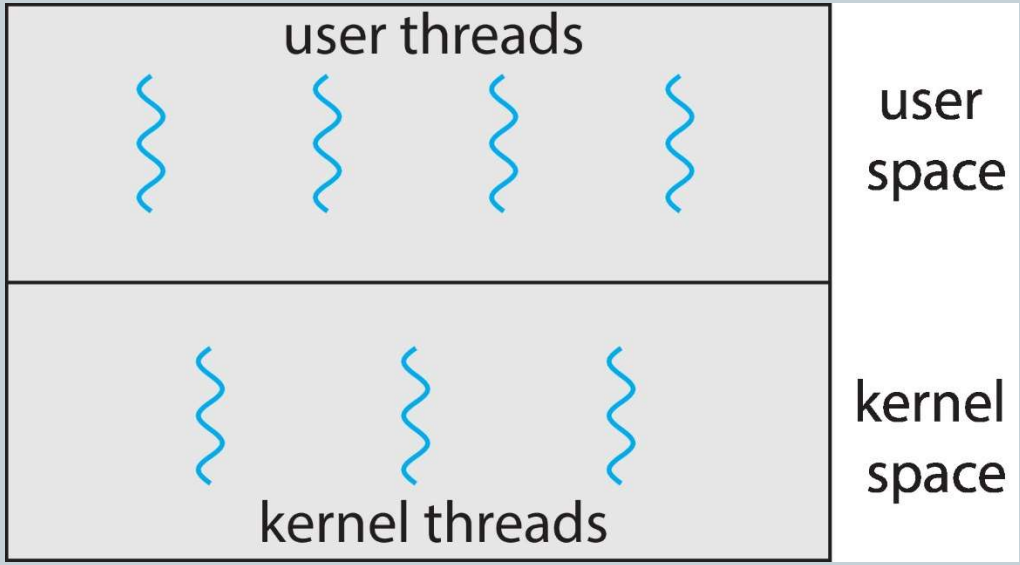


Single and Multithreaded Processes



User Threads and Kernel Threads

- User threads - management done by user-level
- Kernel threads - Supported by the Kernel



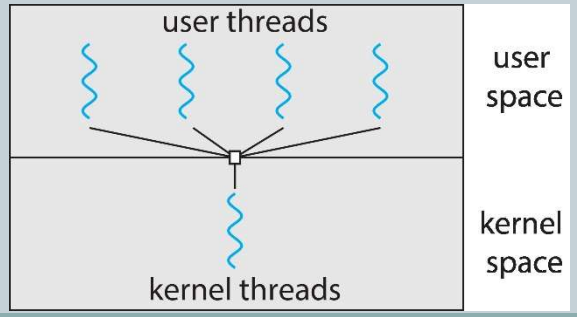
Multithreading Models



- Many-to-One
- One-to-One
- Many-to-Many

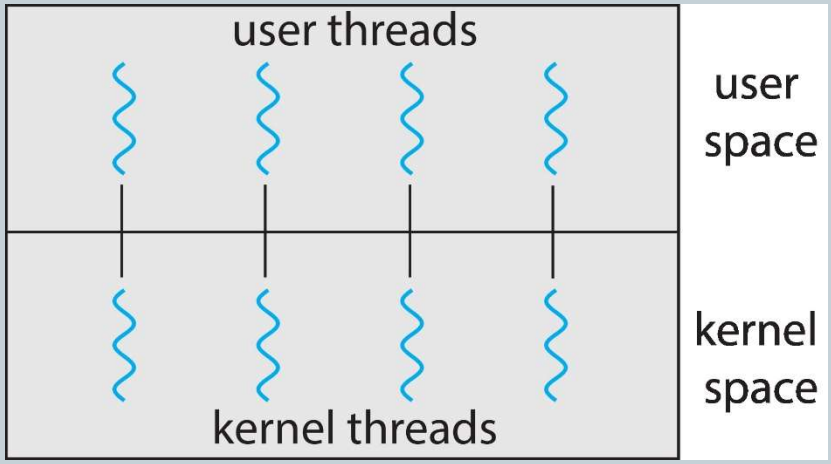
Many-to-One

- Many user-level threads mapped to single kernel thread
- Multiple threads may not run in parallel on muticore system because only one may be in kernel at a time
- Few systems currently use this model



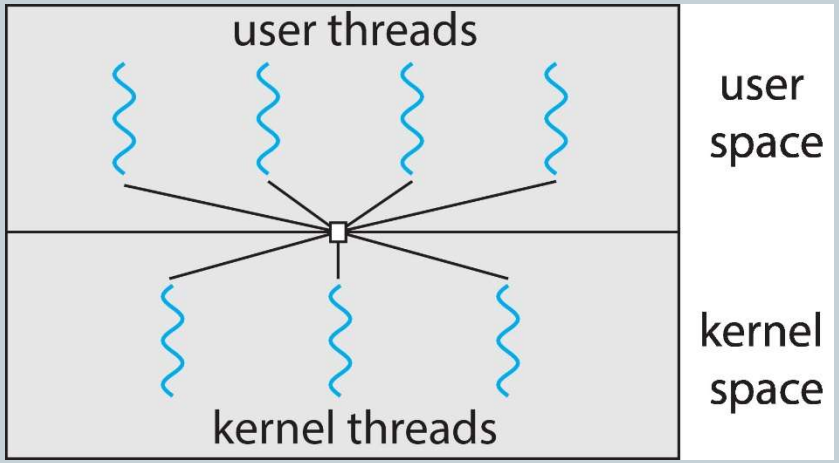
One-to-One

- Each user-level thread maps to kernel thread
- More concurrency than many-to-one
- Number of threads per process restricted due to overhead



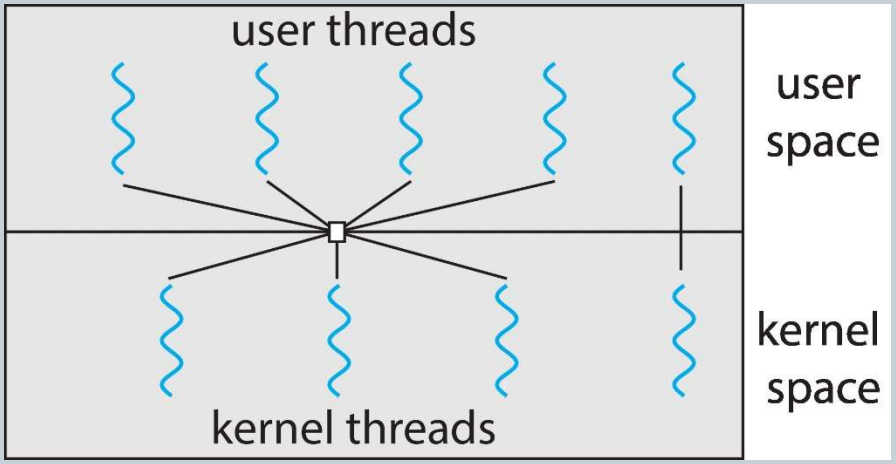
Many-to-Many Model

- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads



Two-level Model

- Similar to M:M, except that it allows a user thread to be **bound** to kernel thread



Thread Libraries



- **Thread library** provides programmer with API for creating and managing threads
- Two primary ways of implementing
  - Library entirely in user space
  - Kernel-level library supported by the OS

Java Threads



- Java threads are managed by the JVM
- Typically implemented using the threads model provided by underlying OS
- Java threads may be created by:

```
public interface Runnable
{
    public abstract void run();
}
```

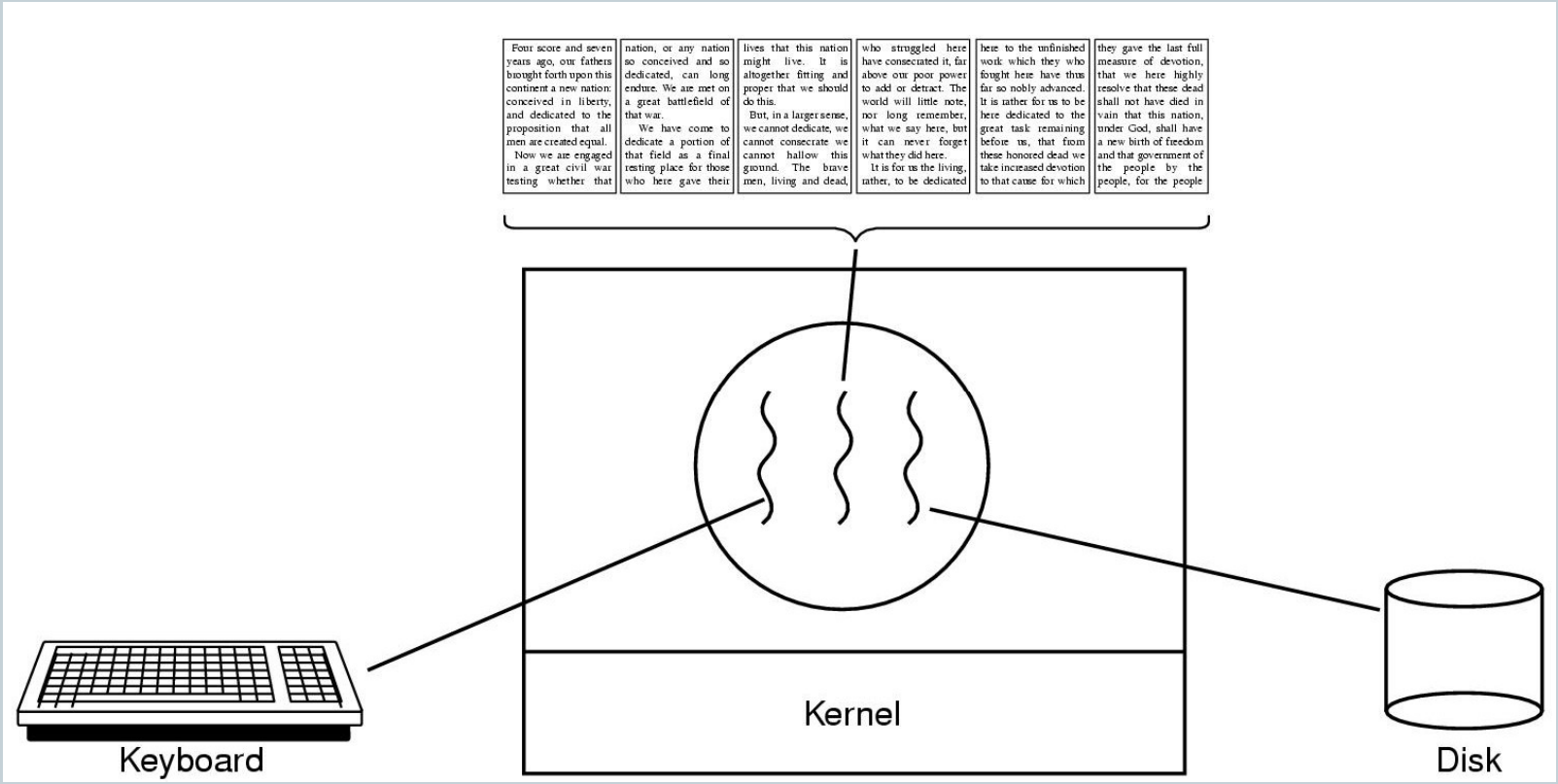
  - Extending Thread class
  - Implementing the Runnable interface

Thread Pools



- Create a number of threads in a pool where they await work
- Advantages:
  - slightly faster to service (request and existing without than create a new thread)
  - Allows the number of threads in the application(s) to be bound to the size of the pool
  - Separating task to be performed from mechanics of creating task allows different strategies for running task

Thread usage



- word processor with three threads